



Plataforma Web de Campanhas D&D 5e

RELATÓRIO DE QUALIDADE DE SOFTWARE

Estratégia de Testes em 3 Níveis

Junho de 2026

107 Testes

100%
Aprovados

90.6%
Cobertura

Nível 1 60 testes
unitários

Nível 2 39 testes
integração

Nível 3 8 testes E2E

Stack: TypeScript

Framework: Jest +
Supertest

9 suítes de teste

1. Visão Geral do Projeto

O EpicForge é uma plataforma web fullstack para gerenciamento de campanhas de D&D 5e. A aplicação oferece CRUD completo de Personagens, Monstros, Builds e Reviews, além de integração com a API oficial D&D 5e API para busca de raças, classes e monstros canônicos.

Componente	Detalhes
Frontend	React 19 + TypeScript + Vite + React Router
Backend	Node.js + Express 5 + TypeScript (armazenamento em memória)
API Externa	D&D 5e API (https://www.dnd5eapi.co/api) — raças, classes e monstros
Testes	Jest 30 + ts-jest + Supertest — cobertura de 3 níveis
Rotas CRUD	/characters, /monsters, /builds, /reviews + /api-external/races classes monsters

1.1 Resumo dos Resultados

Nível de Teste	Suítes	Testes	Passaram	Ferramenta
☐ Nível 1 — Unitários	4	60	60 ☐	Jest + ts-jest
☐ Nível 2 — Integração	4	39	39 ☐	Jest + Supertest
☐ Nível 3 — E2E	1	8	8 ☐	Jest + Supertest
☐ TOTAL GERAL	9	107	107 ☐ (100%)	Jest

2. Nível 1 — Testes Unitários

O que são: Validam funções, métodos e módulos isolados do sistema, sem dependências externas.

Ferramentas: Jest 30 + ts-jest

Arquivos cobertos: validators.ts, rpgUtils.ts, dndService.ts, setup.spec.ts

2.1 Casos de Teste Unitários (TC-U01 a TC-U60)

ID	Função/Método	Objetivo	Entrada	Esperado	Obtido	Status
TC - U01	<i>isValidEmail()</i>	E-mail com formato clássico válido	<i>isValidEmail</i> ("arthur@email.com")	true	true	☐ Aprovado
TC - U02	<i>isValidEmail()</i>	E-mail com subdomínio	<i>isValidEmail</i> ("mestre@dungeon.rpg.br")	true	true	☐ Aprovado
TC - U03	<i>isValidEmail()</i>	E-mail com pontos antes do @	<i>isValidEmail</i> ("first.last@example.com")	true	true	☐ Aprovado
TC - U04	<i>isValidEmail()</i>	E-mail sem símbolo @	<i>isValidEmail</i> ("arthuremail.com")	false	false	☐ Aprovado
TC - U05	<i>isValidEmail()</i>	E-mail sem domínio após @	<i>isValidEmail</i> ("arthur@")	false	false	☐ Aprovado
TC - U06	<i>isValidEmail()</i>	String vazia	<i>isValidEmail</i> ("")	false	false	☐ Aprovado
TC - U07	<i>isValidEmail()</i>	E-mail com espaço interno	<i>isValidEmail</i> ("arth ur@email.com")	false	false	☐ Aprovado
TC - U08	<i>isValidEmail()</i>	E-mail sem extensão (.com)	<i>isValidEmail</i> ("arthur@email")	false	false	☐ Aprovado
TC - U09	<i>somarAtributos()</i>	Soma padrão de personagem D&D	FOR:15,DES:14,CON:13,INT:12,SAB:10,CAR:8	72	72	☐ Aprovado
TC - U10	<i>somarAtributos()</i>	Todos os atributos = 10	Todos os 6 atributos = 10	60	60	☐ Aprovado
TC	<i>somarAtributos()</i>	Atributos	Todos os 6 atributos = 1	6	6	☐

ID	Função/Método	Objetivo	Entrada	Esperado	Obtido	Status
- U1 1	somarAtributos()	mínimos (todos = 1)	Todos os 6 atributos = 20	120	120	Aprova do
TC - U1 2		Atributos máximos (todos = 20)				☐ Aprova do
TC - U1 3	somarAtributos()	Build assimétrica (Wizard)	FOR:8,DES:14,CON:12,INT:18,SAB:13,CAR:10	75	75	☐ Aprova do
TC - U1 4	calcularModificador()	Atributo 10 → mod 0 (neutro)	calcularModificador(10)	0	0	☐ Aprova do
TC - U1 5	calcularModificador()	Atributo 11 → mod 0	calcularModificador(11)	0	0	☐ Aprova do
TC - U1 6	calcularModificador()	Atributo 12 → mod +1	calcularModificador(12)	1	1	☐ Aprova do
TC - U1 7	calcularModificador()	Atributo 20 → mod +5 (máximo)	calcularModificador(20)	5	5	☐ Aprova do
TC - U1 8	calcularModificador()	Atributo 8 → mod -1	calcularModificador(8)	-1	-1	☐ Aprova do
TC - U1 9	calcularModificador()	Atributo 1 → mod -5 (mínimo)	calcularModificador(1)	-5	-5	☐ Aprova do
TC - U2 0	calcularModificador()	Atributo 15 → mod +2	calcularModificador(15)	2	2	☐ Aprova do
TC - U2 1	isNomeValido()	Nome válido de personagem	isNomeValido("Gandalf")	true	true	☐ Aprova do
TC - U2 2	isNomeValido()	Nome composto com espaços	isNomeValido("Aragorn Filho de Arathorn")	true	true	☐ Aprova do
TC - U2 3	isNomeValido()	String vazia	isNomeValido("")	false	false	☐ Aprova do
TC -	isNomeValido()	Nome com apenas 1	isNomeValido("A")	false	false	☐ Aprova do

ID	Função/Método	Objetivo	Entrada	Esperado	Obtido	Status
U2 4		caractere				do
TC - U2 5	<i>isNomeValido()</i>	Nome com > 50 caracteres	isNomeValido("A".repeat(51))	false	false	☐ Aprova do
TC - U2 6	<i>isNomeValido()</i>	Nome exatamente no mínimo (2 chars)	isNomeValido("A")	true	true	☐ Aprova do
TC - U2 7	<i>isNivelValido()</i>	Nível 1 (inicial)	isNivelValido(1)	true	true	☐ Aprova do
TC - U2 8	<i>isNivelValido()</i>	Nível 20 (máximo D&D)	isNivelValido(20)	true	true	☐ Aprova do
TC - U2 9	<i>isNivelValido()</i>	Nível 0 (inválido)	isNivelValido(0)	false	false	☐ Aprova do
TC - U3 0	<i>isNivelValido()</i>	Nível 21 (acima do máximo)	isNivelValido(21)	false	false	☐ Aprova do
TC - U3 1	<i>isNivelValido()</i>	Nível decimal (1.5)	isNivelValido(1.5)	false	false	☐ Aprova do
TC - U3 2	<i>calcularBonusProficie ncia()</i>	Nível 1 → bônus +2	calcularBonusProficiencia(1)	2	2	☐ Aprova do
TC - U3 3	<i>calcularBonusProficie ncia()</i>	Nível 5 → bônus +3	calcularBonusProficiencia(5)	3	3	☐ Aprova do
TC - U3 4	<i>calcularBonusProficie ncia()</i>	Nível 9 → bônus +4	calcularBonusProficiencia(9)	4	4	☐ Aprova do
TC - U3 5	<i>calcularBonusProficie ncia()</i>	Nível 20 → bônus +6	calcularBonusProficiencia(20)	6	6	☐ Aprova do
TC - U3 6	<i>calcularBonusProficie ncia()</i>	Nível 0 (inválido) → 0	calcularBonusProficiencia(0)	0	0	☐ Aprova do
TC - U3	<i>isRatingValido()</i>	Rating 1 (mínimo)	isRatingValido(1)	true	true	☐ Aprova do

ID	Função/Método	Objetivo	Entrada	Esperado	Obtido	Status
7						
TC - U3 8	<i>isRatingValido()</i>	Rating 5 (máximo)	isRatingValido(5)	true	true	☐ Aprova do
TC - U3 9	<i>isRatingValido()</i>	Rating 0 (inválido)	isRatingValido(0)	false	false	☐ Aprova do
TC - U4 0	<i>isRatingValido()</i>	Rating 6 (acima do máximo)	isRatingValido(6)	false	false	☐ Aprova do
TC - U4 1	<i>formatarNome()</i>	Capitalizar nome minúsculo	formatarNome("gandalf")	'Gandalf'	'Gandalf'	☐ Aprova do
TC - U4 2	<i>formatarNome()</i>	Normalizar tudo maiúsculo	formatarNome("ARAGORN")	'Aragorn'	'Aragorn'	☐ Aprova do
TC - U4 3	<i>formatarNome()</i>	Nome composto formatado	formatarNome("legolas greenleaf")	'Legolas Greenleaf'	'Legolas Greenleaf'	☐ Aprova do
TC - U4 4	<i>formatarNome()</i>	Remover espaços nas extremidades	formatarNome(" Gimli ")	'Gimli'	'Gimli'	☐ Aprova do
TC - U4 5	<i>calcularHpInicial()</i>	Barbarian CON 10 → HP 12	calcularHpInicial("barbarian",10)	12	12	☐ Aprova do
TC - U4 6	<i>calcularHpInicial()</i>	Wizard CON 14 → HP 8	calcularHpInicial("wizard",14)	8	8	☐ Aprova do
TC - U4 7	<i>calcularHpInicial()</i>	Fighter CON 8 → HP 9	calcularHpInicial("fighter",8)	9	9	☐ Aprova do
TC - U4 8	<i>calcularHpInicial()</i>	Classe desconhecida → dado d8	calcularHpInicial("cavaleiro",10)	8	8	☐ Aprova do
TC - U4 9	<i>isCrValido()</i>	CR 0 (monstro trivial)	isCrValido(0)	true	true	☐ Aprova do
TC - U5	<i>isCrValido()</i>	CR 30 (monstro lendário)	isCrValido(30)	true	true	☐ Aprova do

ID	Função/Método	Objetivo	Entrada	Esperado	Obtido	Status
0						
TC - U5 1	<i>isCrValido()</i>	CR negativo	isCrValido(-1)	false	false	☐ Aprovado
TC - U5 2	<i>getRaces()</i>	Retorna lista de raças D&D	Mock: count:9, results:[Human...]	count:9, results:array	count:9, results:array	☐ Aprovado
TC - U5 3	<i>getRaces()</i>	Erro se API indisponível	Mock: throw "Network Error"	throws Error	throws Error	☐ Aprovado
TC - U5 4	<i>getRaces()</i>	Chama endpoint /races	Mock chamada	get("/races")	get("/races")	☐ Aprovado
TC - U5 5	<i>getClasses()</i>	Retorna lista de classes	Mock: count:12, results:[Wizard...]	count:12	count:12	☐ Aprovado
TC - U5 6	<i>getClasses()</i>	Chama endpoint /classes	Mock chamada	get("/classes")	get("/classes")	☐ Aprovado
TC - U5 7	<i>getClasses()</i>	Erro se API falhar	Mock: throw "Timeout"	throws Error	throws Error	☐ Aprovado
TC - U5 8	<i>getExternalMonsters()</i>	Retorna lista de monstros	Mock: count:332, results:[Beholder...]	count:332	count:332	☐ Aprovado
TC - U5 9	<i>getExternalMonsters()</i>	Chama endpoint /monsters	Mock chamada	get("/monsters")	get("/monsters")	☐ Aprovado
TC - U6 0	<i>getExternalMonsters()</i>	3 métodos são independentes	Chamadas em sequência	3 calls separadas	3 calls separadas	☐ Aprovado

3. Nível 2 — Testes de Integração

O que são: Verificam a comunicação entre camadas — rotas HTTP, lógica de negócio e persistência em memória.

Ferramentas: Jest 30 + Supertest + Mocks (jest.mock para API externa)

Entidades testadas: Characters (10), Monsters (12), Builds (8), Reviews (8) + API External (3)

3.1 Casos de Teste de Integração (TC-I01 a TC-I38)

ID	Rota/Recurso	Objetivo	Pré-condição / Entrada	Resultado Esperado	Resultado Obtido	Status
TC-I01	POST /characters	Criar personagem válido	name, race, class preenchidos	HTTP 201 + body.id	201 + id gerado	☐
TC-I02	POST /characters	Rejeitar sem campos obrigatórios	Body vazio {}	HTTP 400 + error	400 + error	☐
TC-I03	POST /characters	Rejeitar sem campo name	Apenas race e class	HTTP 400	400	☐
TC-I04	GET /characters	Listar personagens	Rota GET chamada	HTTP 200 + Array	200 + array	☐
TC-I05	GET /characters	Personagem criado aparece na lista	POST + GET	Body contém personagem	Personagem encontrado	☐
TC-I06	PUT /characters/:id	Atualizar personagem existente	ID válido + novos dados	HTTP 200 + dados atualizados	200 + dados corretos	☐
TC-I07	PUT /characters/:id	ID inexistente retorna 404	ID inválido	HTTP 404 + error	404 + error	☐
TC-I08	DELETE /characters/:id	Excluir personagem existente	ID válido	HTTP 204	204	☐
TC-I09	DELETE + GET	Personagem deletado não aparece	DELETE + GET	Body não contém personagem	Personagem ausente	☐
TC-I10	POST /characters	Múltiplos personagens criados	3 CREATEs sequenciais	3 personagens na lista	3 encontrados	☐
TC-I11	POST /monsters	Criar monstro válido	name, type preenchidos	HTTP 201 + body.id	201 + id	☐
TC-I12	POST /monsters	Rejeitar sem campos obrigatórios	Body vazio {}	HTTP 400 + error	400 + error	☐
TC-I13	POST /monsters	Rejeitar sem campo type	Apenas name	HTTP 400	400	☐
TC-I14	GET /monsters	Listar monstros	Rota GET chamada	HTTP 200 + Array	200 + array	☐

ID	Rota/Recurso	Objetivo	Pré-condição / Entrada	Resultado Esperado	Resultado Obtido	Status
TC-I15	GET /monsters	Monstro criado aparece na lista	POST + GET	Body contém monstro	Monstro encontrado	☐
TC-I16	PUT /monsters/:id	Atualizar monstro existente	ID válido + novos dados	HTTP 200 + dados	200 + dados	☐
TC-I17	PUT /monsters/:id	ID inexistente retorna 404	ID inválido	HTTP 404	404	☐
TC-I18	DELETE /monsters/:id	Excluir monstro existente	ID válido	HTTP 204	204	☐
TC-I19	DELETE + GET	Monstro deletado não aparece	DELETE + GET	Body sem monstro	Monstro ausente	☐
TC-I20	GET /api-external/races	Listar raças D&D 5e	Mock dndService	200 + results[9]	200 + 9 raças	☐
TC-I21	GET /api-external/classes	Listar classes D&D 5e	Mock dndService	200 + results[12]	200 + 12 classes	☐
TC-I22	GET /api-external/monsters	Listar monstros D&D 5e	Mock dndService	200 + count:332	200 + 332	☐
TC-I23	POST /builds	Criar build válida	title e author preenchidos	HTTP 201 + body.id	201 + id	☐
TC-I24	POST /builds	Rejeitar sem campos obrigatórios	Body vazio {}	HTTP 400 + error	400 + error	☐
TC-I25	POST /builds	Rejeitar sem campo title	Apenas author	HTTP 400	400	☐
TC-I26	POST /builds	Rating padrão = 0 se não informado	title + author (sem rating)	rating = 0	rating = 0	☐
TC-I27	GET /builds	Listar builds	Rota GET chamada	HTTP 200 + Array	200 + array	☐
TC-I28	GET /builds	Build criada aparece na lista	POST + GET	Body contém build	Build encontrada	☐
TC-I29	DELETE /builds/:id	Excluir build existente	ID válido	HTTP 204	204	☐
TC-I30	DELETE + GET	Build deletada não aparece	DELETE + GET	Body sem build	Build ausente	☐
TC-I31	POST /reviews	Criar review com texto válido	text preenchido	HTTP 201 + id + createdAt	201 + campos	☐
TC-I32	POST /reviews	Rejeitar sem campo text	Body vazio {}	HTTP 400 + error	400 + error	☐
TC-I33	POST /reviews	Rejeitar text vazio ("")	text: ""	HTTP 400	400	☐
TC-I34	POST /reviews	createdAt em formato ISO 8601	POST válido	Data ISO válida	Data válida	☐
TC-I35	GET /reviews	Listar reviews	Rota GET chamada	HTTP 200 + Array	200 + array	☐
TC-	GET /reviews	Review criada	POST + GET	Body contém	Review	☐

ID	Rota/Recurso	Objetivo	Pré- condição / Entrada	Resultado Esperado	Resultado Obtido	Status
I36		aparece na lista		review	encontrada	
TC- I37	<i>DELETE</i> <i>/reviews/:id</i>	Excluir review existente	ID válido	HTTP 204	204	☐
TC- I38	<i>DELETE</i> + <i>GET</i>	Review deletada não aparece	DELETE + GET	Body sem review	Review ausente	☐

4. Nível 3 — Testes End-to-End (E2E)

O que são: Simulam jornadas reais do usuário percorrendo múltiplas rotas em sequência, validando fluxos de ponta a ponta.

Ferramentas: Jest 30 + Supertest (API E2E) + Mocks de API externa

Jornadas testadas: (J1) CRUD completo de personagem | (J2) Campanha completa | (J3) API externa → criação | (J4) Resiliência a erros

4.1 Casos de Teste E2E (TC-E01 a TC-E08)

ID	Jornada	Cenário	Passos de Execução	Resultado Esperado	Resultado Obtido	Status
TC-E01	Jornada 1	Fluxo completo CRUD de personagem	GET → POST → GET → PUT → GET → DELETE → GET	Personagem criado, editado e removido da lista	Fluxo completo executado com sucesso	☐
TC-E02	Jornada 2	Mestre monta sessão de campanha completa	POST monsters×2 → GET → POST build → POST reviews×2 → GET → DELETE×5 → GET	Todos os recursos criados, consultados e removidos	Campanha completa gerenciada	☐
TC-E03	Jornada 3	Criar personagem com dados reais da API D&D	GET /api-external/races → GET /api-external/classes → POST /characters	Personagem com raça e classe da API oficial	Personagem criado com dados da API	☐
TC-E04	Jornada 4	Sistema rejeita dados inválidos em todas as entidades	POST vazio para todas as rotas	HTTP 400 em cada rota	400 retornado em todas	☐
TC-E05	Jornada 4	PUT em IDs inexistentes retorna 404	PUT /characters + PUT /monsters com IDs falsos	HTTP 404 em ambos	404 em ambas as rotas	☐
TC-E06	Jornada 4	DELETE de ID inexistente é idempotente (204)	DELETE /characters com ID fantasma	HTTP 204	204	☐
TC-E07	Jornada 4	Múltiplos recursos criados em paralelo	Promise.all com 3 POSTs simultâneos	201 em todos, IDs únicos	Criação paralela sem conflito	☐
TC-E08	Jornada 4	API externa retorna estrutura consistente	Promise.all com 3 GETs externos	Todos com count e results	Estrutura validada em todas	☐

5. Relatório de Cobertura de Código

Cobertura medida pelo Jest com Istanbul. Meta mínima exigida: 70%. Resultado final: 90.6% de statements cobertos.

Arquivo	Statements	Branches	Functions	Lines
server.ts	87.6%	95.2%	95.8%	87.0%
dndService.ts	100%	100%	100%	100%
validators.ts	100%	100%	100%	100%
rpgUtils.ts	100%	100%	100%	100%
TOTAL	90.6%	97.5%	97.4%	90.1%

5.1 Interpretação dos Resultados

Statements	90.6% — Supera a meta de 70%. As linhas não cobertas são tratativas de erro em rotas que não dispararam exceção.
Branches	97.5% — Quase toda a lógica de desvio foi exercitada, incluindo validações de campos obrigatórios.
Functions	97.4% — Praticamente todas as funções exportadas foram invocadas nos testes.
Lines	90.1% — Compatível com os statements; linhas não cobertas são cláusulas catch de rotas.
dndService.ts	100% em todos os indicadores — cobertura total por meio de mocks de axios.
validators.ts	100% em todos os indicadores — todas as funções testadas com casos válidos e inválidos.
rpgUtils.ts	100% em todos os indicadores — novo módulo de regras D&D com cobertura total.

6. Métricas de Qualidade Estática

Análise realizada com TypeScript Compiler (tsc --noEmit) e ESLint, verificando code smells, complexidade e vulnerabilidades.

Métrica	Observação	Status
Erros de Tipo (TypeScript)	0 erros de tipagem detectados (tsc --noEmit)	☐ PASS
Code Smells	Funções modulares com responsabilidade única; duplicação mínima.	☐ PASS
Complexidade Ciclomática	server.ts tem lógica linear. rpgUtils e validators têm CC < 3.	☐ BAIXA
Vulnerabilidades	Sem input sanitization para SQL (backend usa in-memory, sem SQL injection risk).	☐ PASS
Duplicação de Código	Padrão de rotas repetido em server.ts é estrutural, não acidental. Candidato a refatoração com Router.	☐ BAIXA
Cobertura de Testes	90.6% statements — supera a meta de 70-100% recomendada pelo projeto.	☐ 90.6%

7. Conclusão

A estratégia de testes em 3 níveis foi implementada com êxito no projeto EpicForge, atingindo 107 casos de teste — todos aprovados — com cobertura de código de 90.6%, superando a meta mínima de 70% estabelecida no projeto.

□

Pontos Fortes

- 107 testes criados e 100% aprovados
- 90.6% de cobertura de código
- dndService.ts, validators.ts e rpgUtils.ts com 100%
- 3 níveis distintos e bem delimitados
- E2E cobre 4 jornadas reais de usuário
- Mocks corretos para API externa
- Testes independentes e reproduzíveis

□

Melhorias Futuras

- Adicionar Playwright para E2E de browser real
- Cobrir as linhas catch de server.ts
- Configurar SonarQube para análise contínua
- Implementar banco real (Prisma/PostgreSQL)
- Adicionar testes de carga com k6 ou Artillery
- CI/CD com GitHub Actions rodando testes
- Adicionar autenticação e testes de segurança